

基于TLV协议的数据存储服务系统

2025年6月9日 9:28

框架（线程池）+ 业务

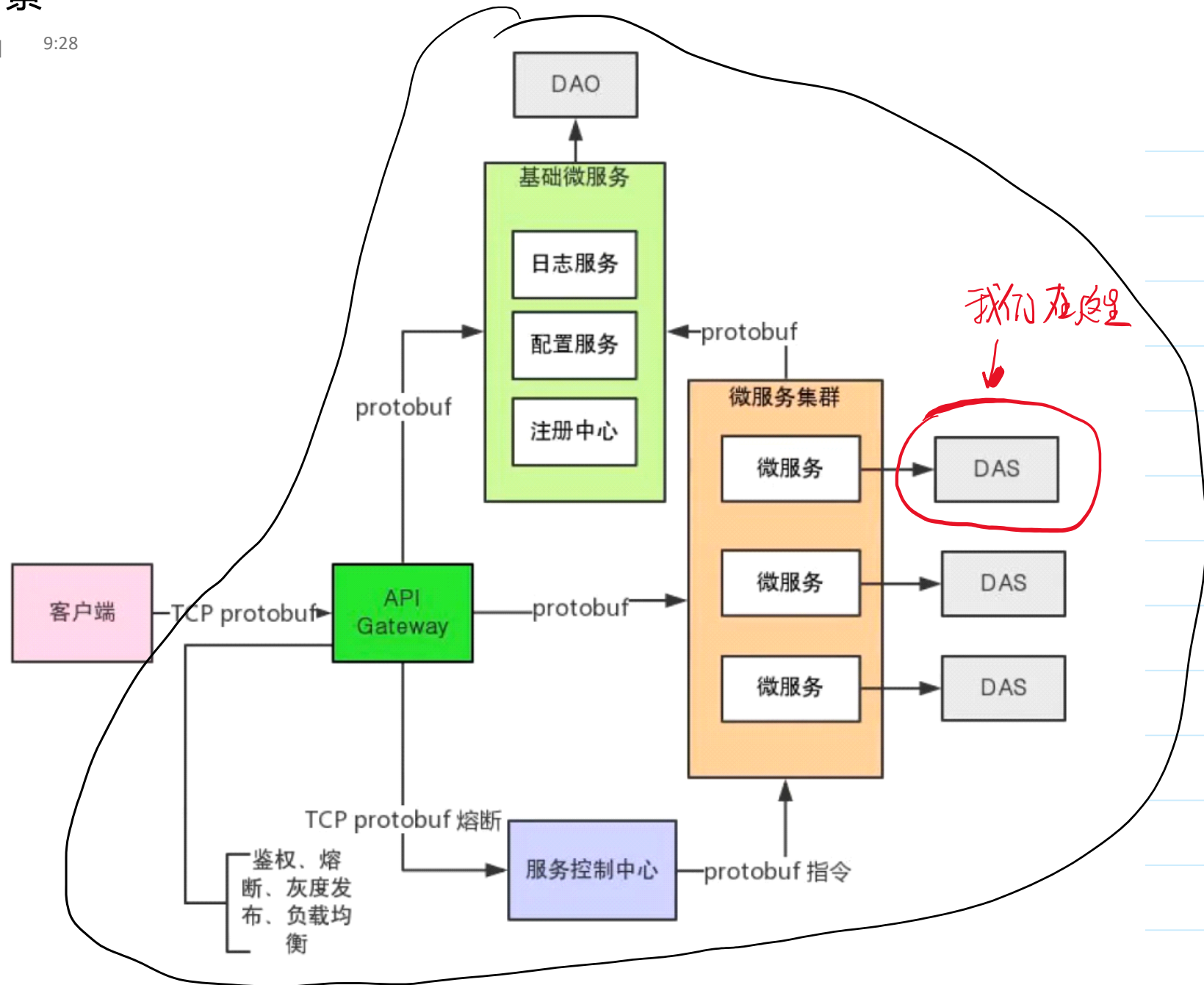
多期：

- 1、1期&&2期 实现网盘的基本功能
- 2、3期 进入MySQL，重构网盘
- 3、4期 重写服务端框架层
- 4、5期
 - 单机改成多服务端
 - 多协议支持
 - P2P

小步快跑，项目不是写出来的，项目是改出来的

业务背景

2025年6月9日 9:28

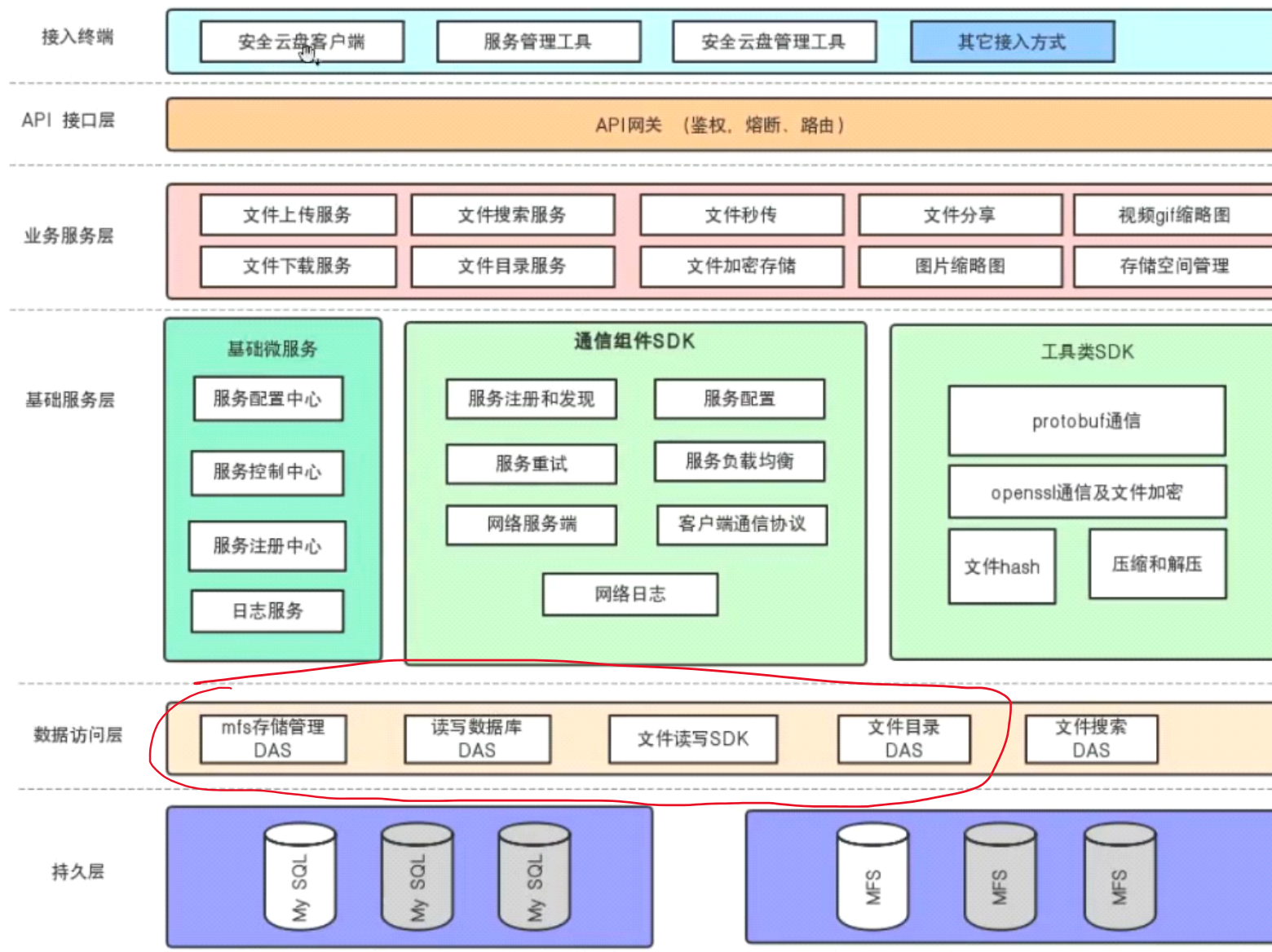


层次图

2025年6月9日

9:29

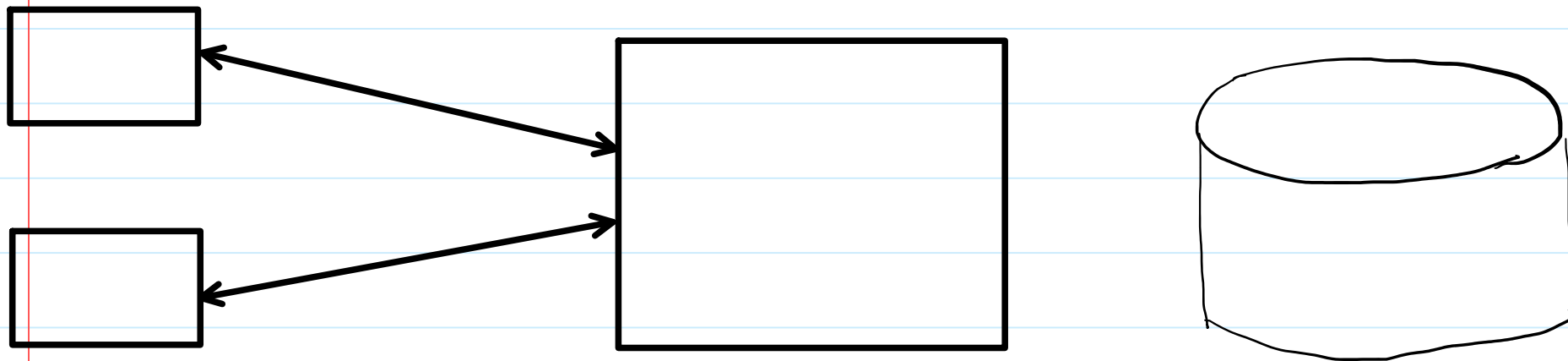
安全云盘微服务系统架构图



1期和2期需求

2025年6月9日 9:28

1期是一个非常简陋的网盘，没有注册和登录功能，我们聚焦于命令处理和数据读写



命令

2025年6月9日 9:57

文件读写: puts gets remove

目录操作: cd ls pwd mkdir
rmdir

1. cd 进入对应目录
2. ls 列出相应目录文件
3. puts 将本地文件上传至服务器
4. gets 文件名 下载服务器文件到本地
5. remove 删除服务器上文件
6. pwd 显示目前所在路径
7. 其他命令可以自己添加 (mkdir 等等)
8. 无效命令不响应



真目录 还是假目录?

用户接触的目录 —— 虚拟目录

2025年6月9日 9:59



/home/liao/netdisk/user1/C++46/static

用户操作的是虚拟目录，需要通过字符串拼接转成真实目录

ls 将虚拟目录转成真实目录，`opendir&&readdir`

pwd 显示虚拟目录给客户端

cd ~~调用chdir~~

cd命令的实现不能用chdir

2025年6月9日 10:05

当前工作目录是进程的属性，如果多个客户端连接到同一个服务端，一个客户端chdir了，另一个客户端的当前工作目录变了

我们实现cd的时候，不需要真的改当前工作目录，只需要让用户以为目录改了就行。

/home/liao/netdisk/user1/C++46/static

↓ ↓
cwd. 参数.

cd dir1 /C++46 --> /C++46/dir1

cd的难点

2025年6月9日 10:10

1、判断子目录是否存在

算出子目录的真实路径，再调用access

```
int access(const char *pathname, int mode);
```

2、

cd ..

cd ../dir1/../dir2

设计一个栈结构.

cd .. 弹栈

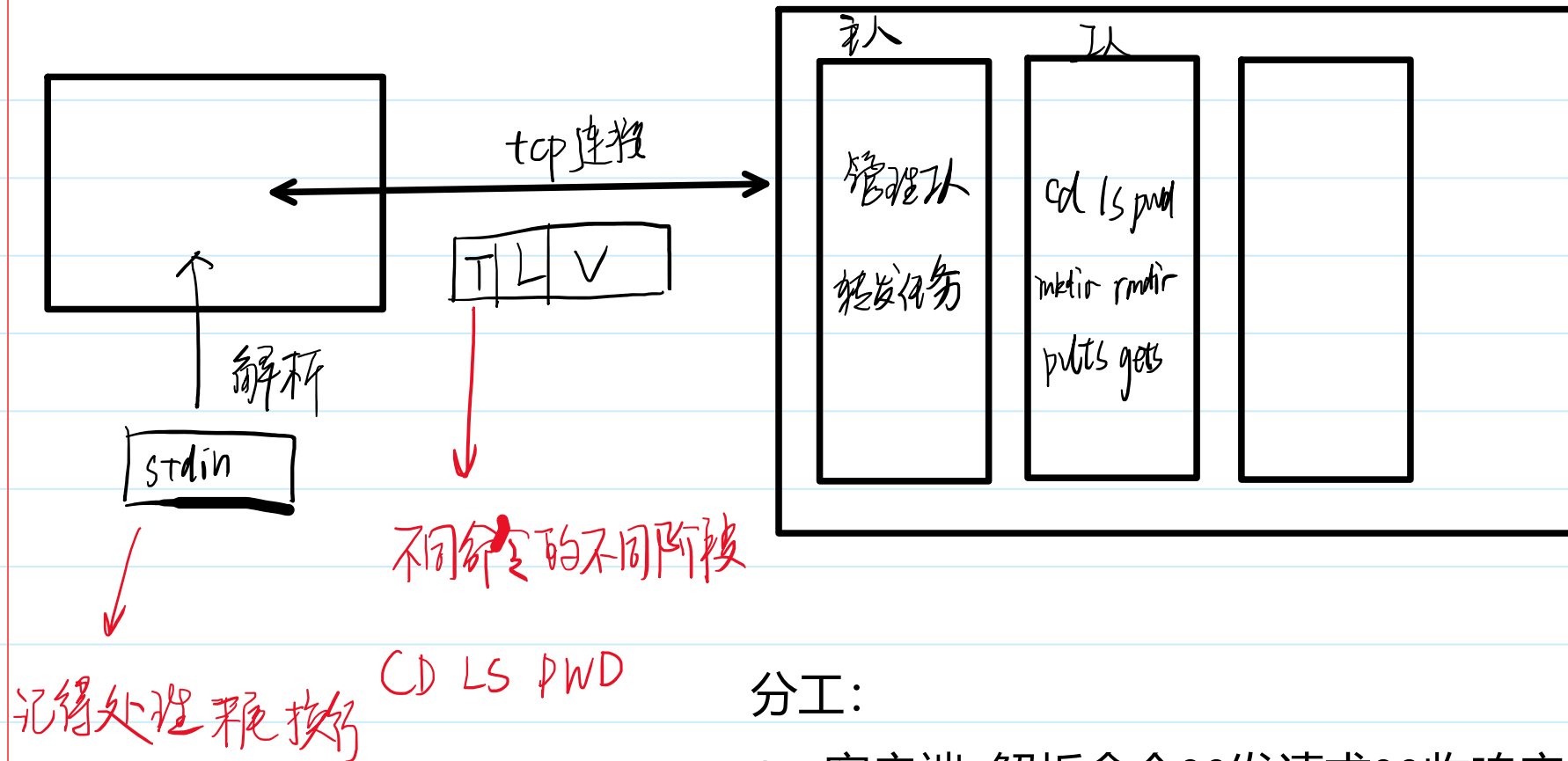
cd 子目录 压栈

pwd 遍历栈

strtok + 循环.

1期的结构

2025年6月9日 10:18



分工:

- 1、客户端 解析命令&&发请求&&收响应
- 2、服务端框架 线程池&&选择具体的服务
- 3、文件上传&&下载
- 4、目录相关的命令 `pwd cd ls mkdir rmdir`

2期

2025年6月9日 10:51

2期是在1期的基础上添加功能

登录

我们在1期和2期没有做注册功能，所以我们只能通过借助Linux的用户，我们直接使用操作系统的用户名和密码，可以登录我们的网盘



网盘想要验证密码，但是网盘不知道密码是多少？

引入了登录之后

2025年6月9日 10:53

1期的前缀要做调整

2期的前缀会根据用户调整.

/home/liao/netdisk/niqu/C++46/static
/home/liao/netdisk/other/C++46/static

shadow文件存储了密文密码

2025年6月9日 10:57

```
7$ sudo vim /etc/shadow
```

```
niqiu:$6$yKUuZc3.zF3o2B0J$Fa1Chinnft6VwCytRCI65h1Y8vQKqkDfC7MBZOhmQZu4tRaxfKGE/IEGqy10yqFKKrsqsT5u/rUnMsrOciVBo/:20035:0:99999:7:::  
other:$6$4siXf1QfpX85B6eF$nkawggBJV0eGW1B69c9T88OpHuLhwaexvLrXmYUBW1E9cJGsP3KZ4fgZuvRNeEtI5YBPTkgrT6y5n2X/BPkJ/:20098:0:99999:7:::
```

盐值

明文 + 盐值 $\xrightarrow{\text{加密}}$ 密文
↓
单向.

服务端已知密文和盐值，用户发来一个新明文，服务端根据新明文和盐值，加密得到新密文，对比新旧密文

计算密文的函数

2025年6月9日

11:02

密文. 明文 盐值
`(char *`
`crypt(const char *phrase, const char *setting);`

链接要加选项

:, -lcrypt)

0_crypt.c

```
1 #include <66func.h>
2 int main(int argc, char *argv[])
3 {
4     char passwd[] = "123";
5     char salt[] = "$6$4siXf1QfpX85B6eF$";
6     printf("encrypted password = %s\n", crypt(passwd,salt));
7     return 0;
8 }
9
```

提取密文

2025年6月9日 11:07

接下来的函数必须在root下执行

```
#include <shadow.h>
```

```
struct spwd *getspnam(const char *name);
```

```
struct spwd {  
    char *sp_namp;      /* Login name */  
    char *sp_pwdp;      /* Encrypted password */  
    long sp_lstchg;     /* Date of last change  
                        (measured in days since  
                        1970-01-01 00:00:00 +0000 (UTC)) */  
    long sp_min;        /* Min # of days between changes */  
    long sp_max;        /* Max # of days between changes */  
    long sp_warn;       /* # of days before password expires  
                        to warn user to change it */  
    long sp_inact;      /* # of days after password expires  
                        until account is disabled */  
    long sp_expire;     /* Date when account expires  
                        (measured in days since  
                        1970-01-01 00:00:00 +0000 (UTC)) */  
    unsigned long sp_flag; /* Reserved */  
};
```

1_getspnam.c

```
1 #include <66func.h>
2 #include <shadow.h>
3 int main(int argc, char *argv[])
4 {
5     struct spwd * userInfo = getspnam("other");
6     ERROR_CHECK(userInfo, NULL, "getspnam");
7     printf("encrypted password = %s\n", userInfo->sp_pwdp);
8     return 0;
9 }
10
```

~

必须要在root用户下面执行

操作系统的日志

2025年6月9日 11:12

```
9$ cat /var/log/syslog
```

```
#include <syslog.h>
```

```
void openlog(const char *ident, int option, int facility);
```

```
void syslog(int priority, const char *format, ...);
```

```
void closelog(void);
```

LOG_EMERG

LOG_ALERT

LOG_CRIT

LOG_ERR

LOG_WARNING

LOG_NOTICE

LOG_INFO

LOG_DEBUG

和 printf 一样。

日志的要求

2025年6月9日 11:17

```
2_syslog.c+
1 #include <66func.h>
2 #include <syslog.h>
3 int main(int argc, char *argv[])
4 {
5     for(int i = 0; i < 10; ++i){
6         syslog(LOG_INFO,"i = %d\n",i);
7         sleep(2);
8     }
9     return 0;
10 }
11
```

2.日志记录

- 1、日志记录客户端请求信息，及客户端连接时间
- 2、日志记录客户端操作记录，及操作时间

把syslog函数的使用，封装成类似于 ERROR_CHECK这种带参数的宏定义

一些编译器的后门

2025年6月9日 11:18

3_macro.c+

```
1 #include <66func.h>
2 int main(int argc, char *argv[])
3 {
4     printf("__LINE__ = %d\n", __LINE__);
5     printf("__FUNCTION__ = %s\n", __FUNCTION__);
6     printf("__FILE__ = %s\n", __FILE__);
7
8     return 0;
9 }
10
```

断点续传

2025年6月9日 11:24

3.断点续传

1、进行 `gets hello.avi` 时候，判断本地是否存在 `hello.avi`，如果存在，`stat` 获取 `hello.avi` 的大小，传递给服务器的是 `gets hello.avi`

简单的方案：

如果本地发现重名文件，则获取已经下载的大小，通信双方同时做 `lseek`，再传 ---> 传完之后，对比哈希值

复杂方案：

准备一个额外的文件，记录目标已经传输的字节数

零拷贝

2025年6月9日

11:28

4.mmap将大文件映射入内存，进行网络传递

1、当你发现文件大于100M，就使用 `mmap` 该文件，然后再传输

如果可以的话，优先使用 `sendfile`

`mmap/sendfile` 和 `lseek` 有冲突

小组如何合作

2025年6月9日 11:32

1、组会：

- 需求
- 分工（由组长决定）
- 讨论技术细节

2、定接口的注意事项

- 函数声明 先定传入参数，再定传入传出，返回值一律int
- 客户端和服务端要交互的数据格式，服务端拍板

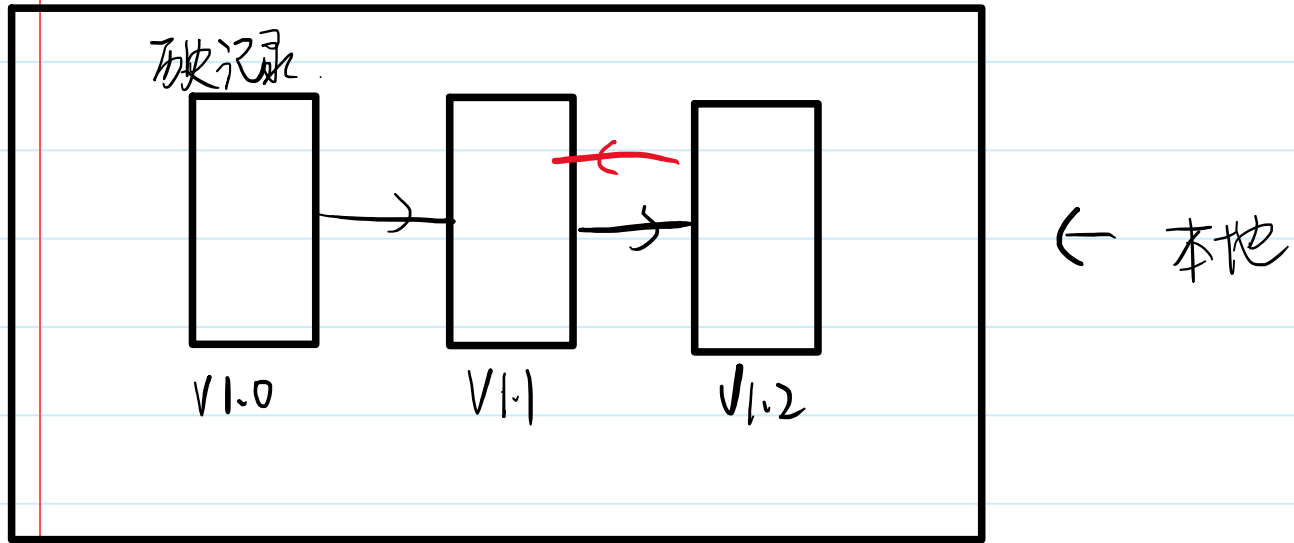
3、记得增量编写法，多做单元测试

4、有问题卡住了，一定要通知到位

5、开了组会必须要有记录（在线文档）

git 版本控制系统

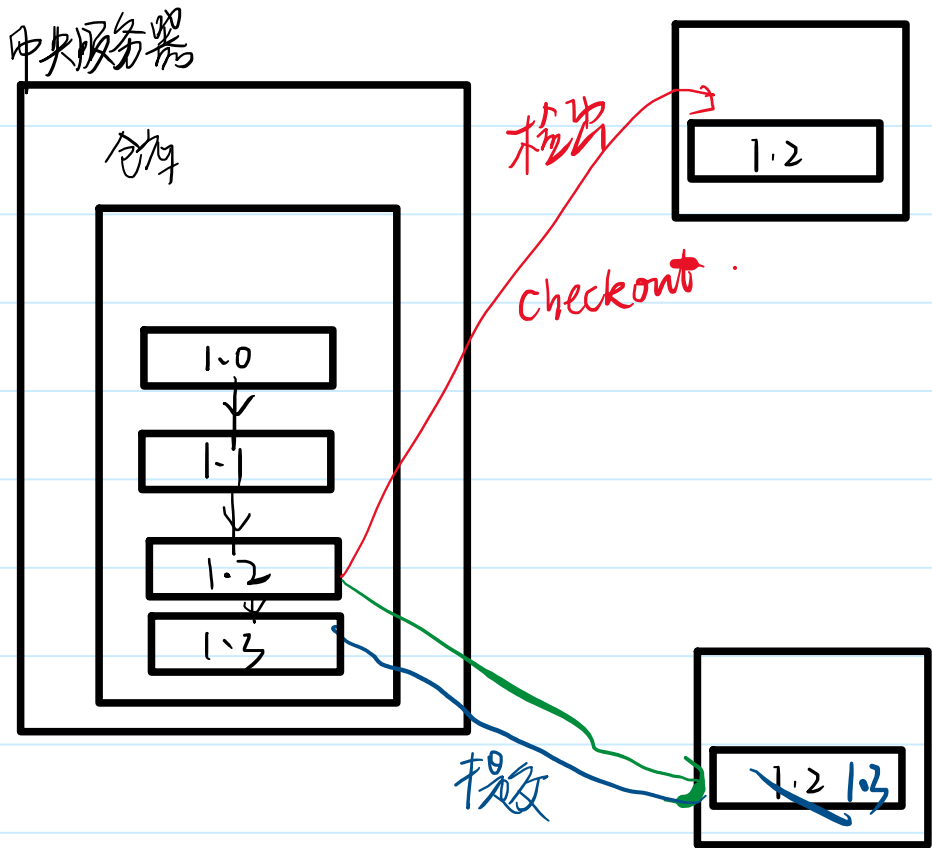
2025年6月9日 11:46



仓库.

集中式

2025年6月9日 11:51

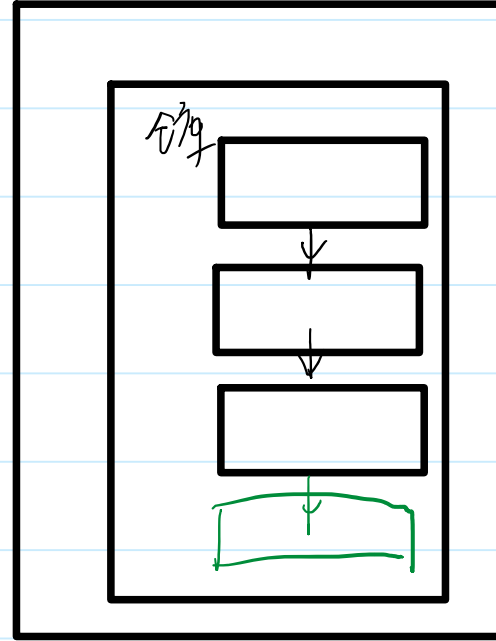
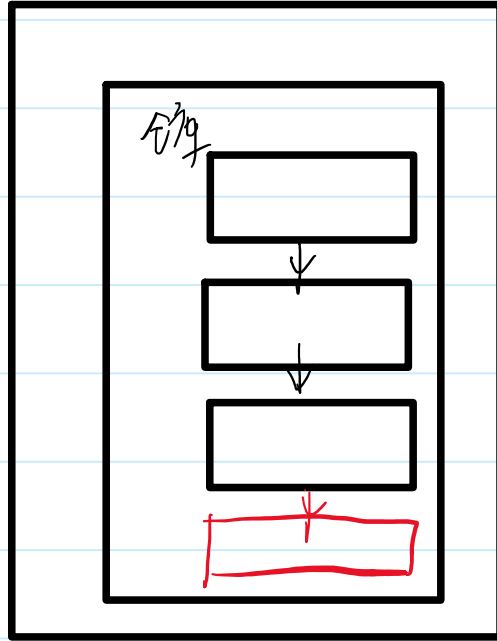


SVN

分布式

2025年6月9日

11:51



git

使用git的步骤

2025年6月9日 14:37

1、申请一个gitee账号(也可以用GitHub)

2、在本地安装git工具

```
~$ sudo apt install git
```

3、

```
git config --global user.name "John Doe"
git config --global user.email johndoe@example.com
```

配的名字和邮箱。

```
git config --global core.editor vim
```

配置密钥

2025年6月9日 14:39

```
liao:~$ ssh-keygen
```

生成一对公钥和私钥 ---> 身份验证

```
liao:~$ ll ~/.ssh/
total 24
drwx----- 2 liao liao 4096 Mar  6  2023 ./
drwxr-xr-x 38 liao liao 4096 Jun  9 11:21 ../
-rw-rw-r-- 1 liao liao  576 Dec  8  2022 authorized_keys
-rw----- 1 liao liao 2590 Mar  6  2023 id_rsa ← 私钥
-rw-r--r-- 1 liao liao  565 Mar  6  2023 id_rsa.pub ← 公钥
-rw-r--r-- 1 liao liao 2442 Jan 14 17:45 known_hosts
```

```
liao:~$ cat ~/.ssh/id_rsa.pub
ssh-rsa AAAAB3NzaC1yc2EAAAADAQABAAQgQCtZcwjwkBa5d+KazNaLx6/bUMygUEoxT
Rn9VfYFOZPjec+spFcCsdWIZuLix0UECEEEi0rnRfUnKH1auScs85jKLUnIM6u0dmXHM9+
cI3OM0ecs21uqc4twaxnS+APV8aQ70qPW/kLUzyG4iD7D63wbYE7xdZeLYSjBt/CkCJaD+
J0FOCzJl3/XH7Iqshohm1NYCH63gfFE+/5R4TQi21fSfGJ/j3pjLW60jqr1h0Nw0PB9kaJ
zqvaLfL7T9IiISgXKhVoOX51SzCExu0zCR4Esc1J9DAVAaQ15QcYB2zAoV0Jx71QY7YrUG
P3Lh9QMNT6NsWXaj+x/ByKZgCu8LzOvQRL0vKI2tWo1wnkBaD1c5EV1lSmwTnGKUHHMBqz
EtspiyMZSXJKAi4sY3v0ouRs2QGwRT9Sn2vUziS9ZU2kQF0twgILE7i+ZTSw2u27JQ1UhV
1JvvV682CNHvAnRTOZ5sBK6bTwoxwTxYXikV14j8x+XQnmVIT8YeHtceAMDwc= liao@ub
untu
liao:~$
```

将公钥放入账户信息中

2025年6月9日 14:43



我的投递地址

代码风格

仓库首选项

安全设置

SSH 公钥

GPG公钥

配置公钥之后：Linux的用户已经和gitee的用户关联在一起了

在云端去创建一个仓库

2025年6月9日 14:39

廖正松/niqui_test_66

代码

Issues 0

Pull Requests 0

Wiki

统计

流水线

服务

管理

master

分支 1 标签 0

+

克隆/下载

廖正松 Initial commit f9e497e 刚刚

1 次提交

.gitignore

Initial commit

刚刚

LICENSE

Initial commit

刚刚

克隆/下载

×

HTTPS

SSH

SVN

SVN+SSH

下载ZIP

git@gitee.com:liao_zheng_song/niqui_test_66.git

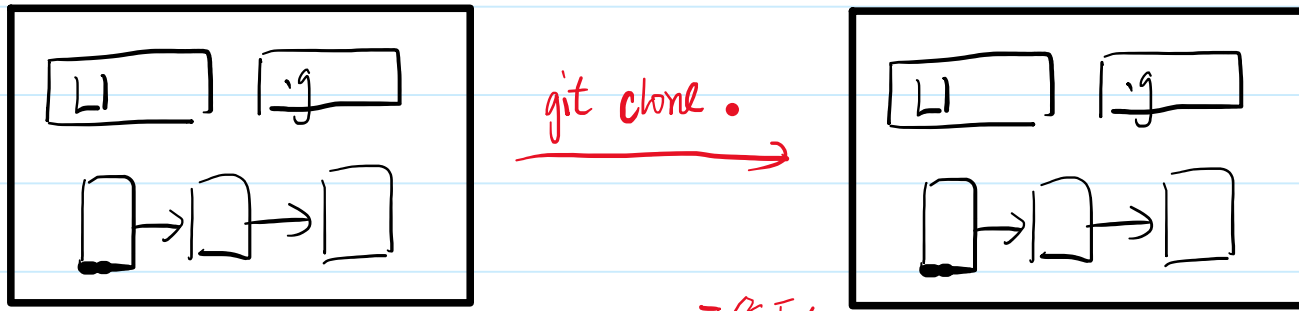
📄

提示

git clone把云端的仓库克隆到本地

2025年6月9日 14:50

```
git clone git@gitee.com:liao_zheng_song/niqu_test_66.git niqu_test_66
```



git clone不只复制了仓库的文件，还复制了历史记录

`.git/`

后续的所有指令

2025年6月9日 14:54

需要先将当前工作目录，切换到仓库内部

```
liao:niqu_test_66$ git status
```

```
On branch master
```

```
Your branch is up to date with 'origin/master'.
```

```
nothing to commit, working tree clean
```

```
liao:niqu_test_66$ cd ..
```

```
liao:~$ git status
```

```
fatal: not a git repository (or any of the parent directories): .git
```

在仓库中新建了一个文件

2025年6月9日 14:58

```
liao:niqu_test_66$ touch main.c
liao:niqu_test_66$ git status
On branch master
Your branch is up to date with 'origin/master'.

Untracked files:
  (use "git add <file>..." to include in what will be committed)
    main.c

nothing added to commit but untracked files present (use "git add" to track)
liao:niqu_test_66$ █
```

untracked 不被追踪

一般 .o文件/可执行程序/测试文件 要一直保持untracked

git add

2025年6月9日 15:03

```
liao:niqu_test_66$ git add main.c
```

```
liao:niqu_test_66$ git status
```

On branch master

Your branch is up to date with 'origin/master'.

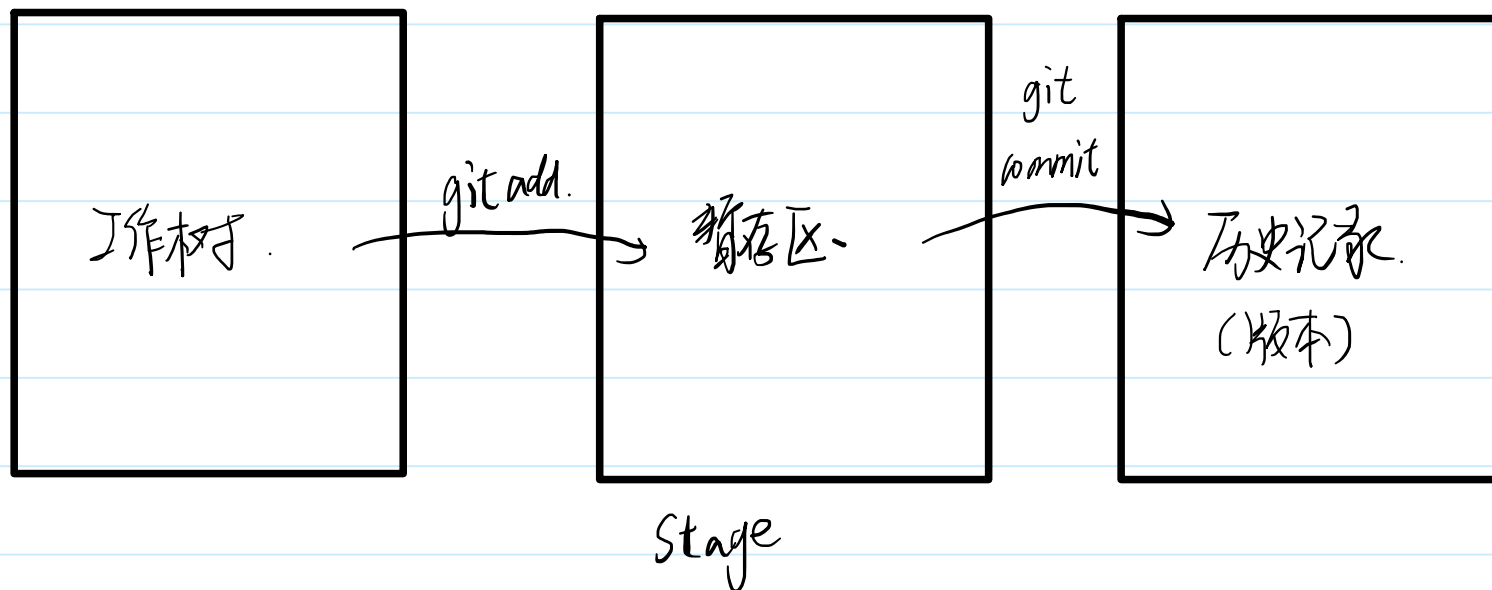
Changes to be committed:

(use "git restore --staged <file>..." to unstage)

new file: main.c

将工作树的文件放入暂存

区



这个命令不要主动使用

2025年6月9日 15:07

先 git status

```
restored you must specify path(s) to restore  
liao:niqui_test_66$ git restore --staged main.c  
liao:niqui_test_66$ git status  
On branch master  
Your branch is up to date with 'origin/master'.
```

Untracked files:

(use "git add <file>..." to include in what will be committed)
main.c

nothing added to commit but untracked files present (use "git add" to track)

git commit

2025年6月9日 15:09

```
liao:niqui_test_66$ git commit -m "20250609 niqiu add main.c"
[master 98ca856] 20250609 niqiu add main.c
1 file changed, 0 insertions(+), 0 deletions(-)
create mode 100644 main.c
```

```
create mode 100644 main.c
```

```
liao:niqui_test_66$ git status
```

On branch master

Your branch is ahead of 'origin/master' by 1 commit.

(use "git push" to publish your local commits)

```
nothing to commit, working tree clean
```

每一次commit会多一个历史记录

2025年6月9日 15:12

```
liao:niqiu_test_66$ git log
commit 98ca85631070f3207a992543eea6e5b3f8675c03 (HEAD -> master)
Author: Liao Zhengsong <jereok@sina.com>
Date:   Mon Jun 9 15:10:25 2025 +0800

    20250609 niqiu add main.c

commit f9e497e224dbe54ce6ce794d01a6f84f177987d6 (origin/master, origin/HEAD)
Author: 廖正松 <jereok@sina.com>
Date:   Mon Jun 9 06:47:11 2025 +0000

    Initial commit
```



提交完成之后 修改文件

2025年6月9日 15:17

```
-----  
liao:niqui_test_66$ cat main.c  
#include <stdio.h>  
int main(){  
    printf("Hello world!\n");  
    return 0;  
}
```

```
liao:niqui_test_66$ git status
```

On branch master

Your branch is ahead of 'origin/master' by 1 commit.
(use "git push" to publish your local commits)

Changes not staged for commit:

(use "git add <file>..." to update what will be committed)

(use "git restore <file>..." to discard changes in working directory)

modified: main.c

no changes added to commit (use "git add" and/or "git commit -a")

```
liao:niqui_test_66$ git restore main.c
```

```
liao:niqui_test_66$ cat main.c
```

```
liao:niqui_test_66$ █
```

← 重新修改

加入暂存区

2025年6月9日 15:20

```
liao:niqu_test_66$ git add main.c
liao:niqu_test_66$ git status
On branch master
Your branch is ahead of 'origin/master' by 1 commit.
  (use "git push" to publish your local commits)

Changes to be committed:
  (use "git restore --staged <file>..." to unstage)
        modified:   main.c
```

将暂存区的内容放入历史记录

2025年6月9日 15:22

```
liao:niqiu_test_66$ git commit -m "20250609 niqiu modify main.c"
[master 2fbc2b0] 20250609 niqiu modify main.c
1 file changed, 5 insertions(+)
```

```
liao:niqiu_test_66$ git log
commit 2fbc2b03c0c602e9ba91a47c48b18c2fa9b54379 (HEAD -> master)
Author: Liao Zhengsong <jereok@sina.com>
Date: Mon Jun 9 15:21:38 2025 +0800
```

20250609 niqiu modify main.c

```
commit 98ca85631070f3207a992543eea6e5b3f8675c03
Author: Liao Zhengsong <jereok@sina.com>
Date: Mon Jun 9 15:10:25 2025 +0800
```

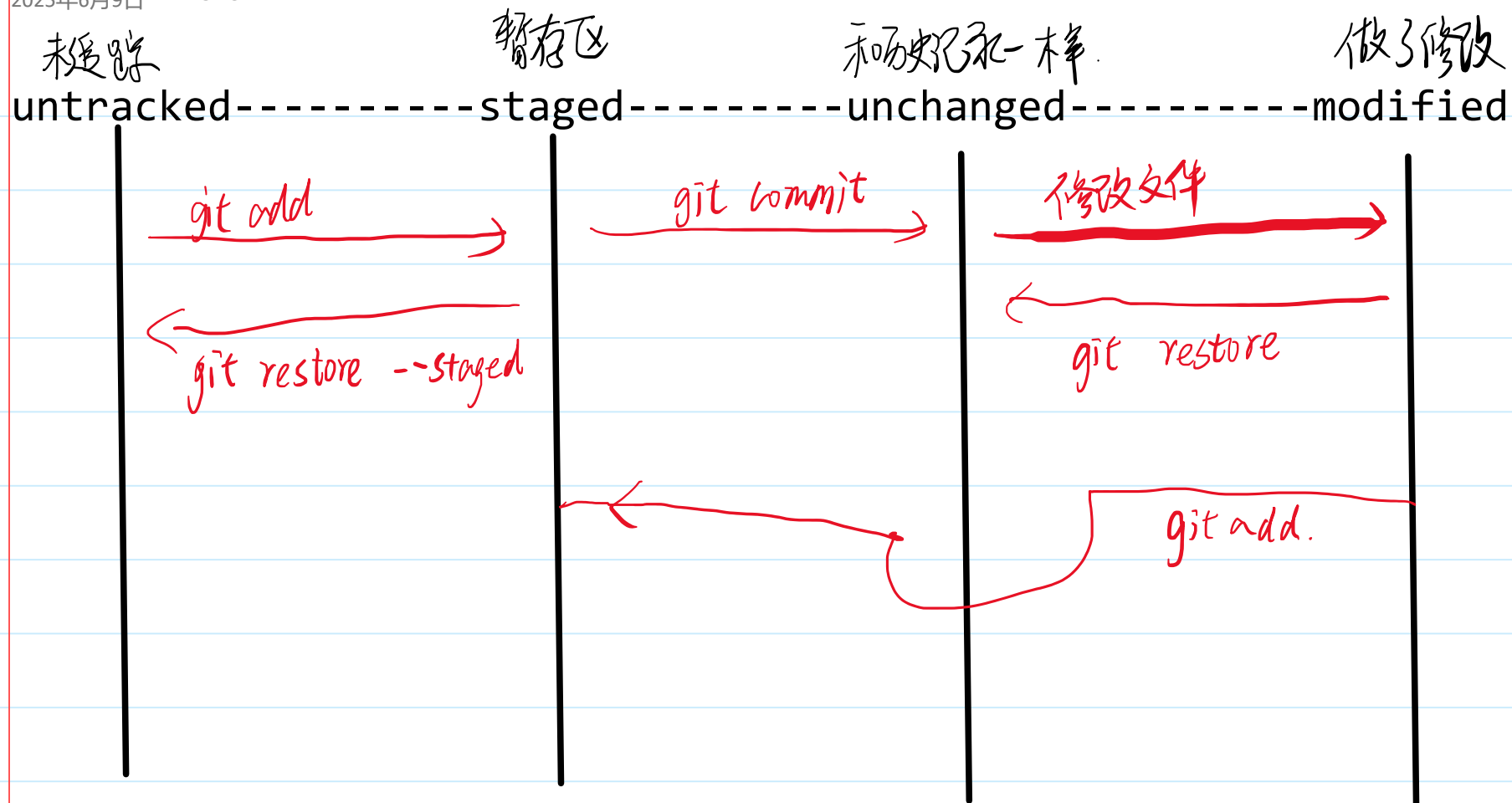
20250609 niqiu add main.c

```
commit f9e497e224dbe54ce6ce794d01a6f84f177987d6 (origin/master, origin/HEAD)
Author: 廖正松 <jereok@sina.com>
Date: Mon Jun 9 06:47:11 2025 +0000
```

Initial commit

文件的状态

2025年6月9日 15:23



时光机

2025年6月9日 15:50

接下来所有命令在执行之前，记得执行一下git status

```
liao:niqui_test_66$ git status
```

```
On branch master
```

```
Your branch is ahead of 'origin/master' by 2 commits.
```

```
(use "git push" to publish your local commits)
```

```
nothing to commit, working tree clean
```

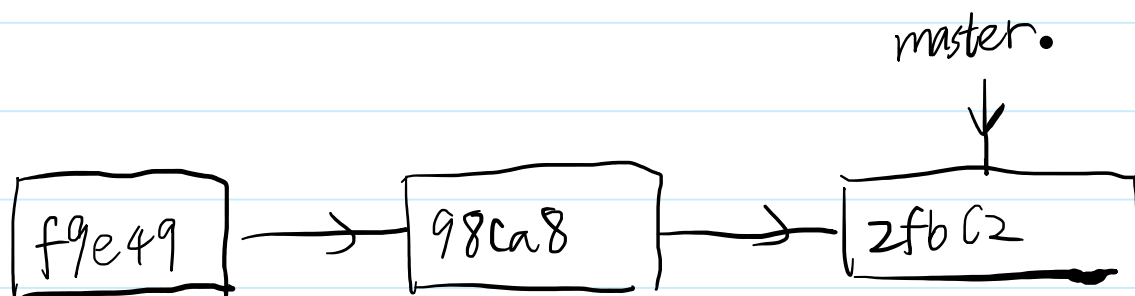
```
liao:niqui_test_66$
```

```
liao:niqui_test_66$ git log --pretty=oneline --graph --all
```

```
* 2fbc2b03c0c602e9ba91a47c48b18c2fa9b54379 (HEAD -> master) 20250609 niqiu modify main.c
```

```
* 98ca85631070f3207a992543eea6e5b3f8675c03 20250609 niqiu add main.c
```

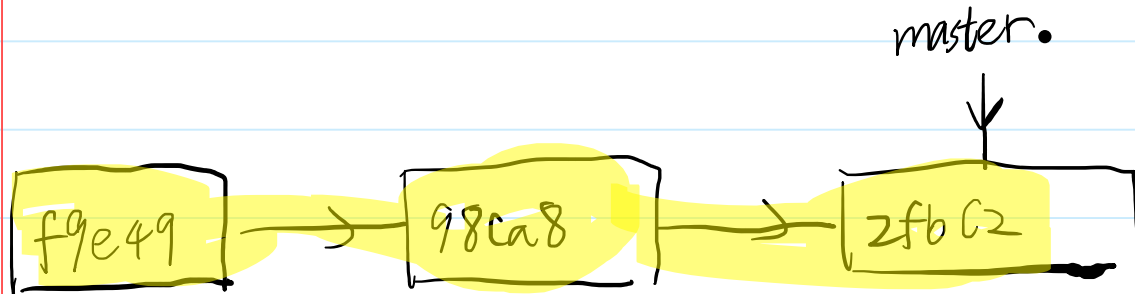
```
* f9e497e224dbe54ce6ce794d01a6f84f177987d6 (origin/master, origin/HEAD) Initial commit
```



分支

2025年6月9日 15:54

分支是一个指针，指向了历史记录当中的某个版本

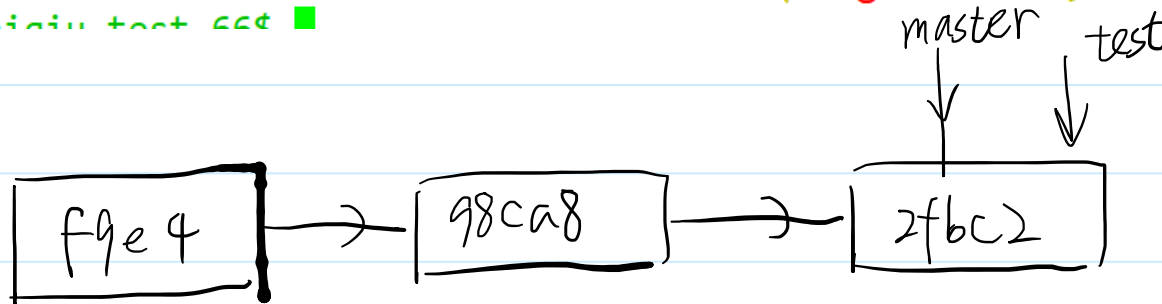


HEAD是一个指针变量，用来记录当前分支

新建分支

2025年6月9日 15:57

```
liao:niqu_test_66$ git branch test
liao:niqu_test_66$ git log --pretty=oneline --graph --all
* 2fbc2b03c0c602e9ba91a47c48b18c2fa9b54379 (HEAD -> master, test) 20250609 niqiu modify main.c
* 98ca85631070f3207a992543eea6e5b3f8675c03 20250609 niqiu add main.c
* f9e497e224dbe54ce6ce794d01a6f84f177987d6 (origin/master, origin/HEAD) Initial commit
liao:niqu_test_66$ █
```



切换分支

2025年6月9日 15:59

```
liao:niqu_test_66$ git checkout test
Switched to branch 'test'
liao:niqu_test_66$ git log --pretty=oneline --graph --all
* 2fbc2b03c0c602e9ba91a47c48b18c2fa9b54379 (HEAD -> test, master) 20250609 niqiu modify main.c
* 98ca85631070f3207a992543eea6e5b3f8675c03 20250609 niqiu add main.c
* f9e497e224dbe54ce6ce794d01a6f84f177987d6 (origin/master, origin/HEAD) Initial commit
```

git commit 是当前分支往后走一格

2025年6月9日 16:01

```
liao:niqui_test_66$ vim main.c
liao:niqui_test_66$ git status
On branch test
Changes not staged for commit:
  (use "git add <file>..." to update what will be committed)
  (use "git restore <file>..." to discard changes in working directory)
        modified:   main.c

no changes added to commit (use "git add" and/or "git commit -a")
liao:niqui_test_66$ git add main.c
liao:niqui_test_66$ git commit -m "test"
[test b1cef02] test
 1 file changed, 1 insertion(+)
liao:niqui_test_66$ git log --pretty=oneline --graph --all
* b1cef02b990abc5f6a47a1a145c169343440bdd7 (HEAD -> test) test
* 2fbc2b03c0c602e9ba91a47c48b18c2fa9b54379 (master) 20250609 niqiu modify main.c
* 98ca85631070f3207a992543eea6e5b3f8675c03 20250609 niqiu add main.c
* f9e497e224dbe54ce6ce794d01a6f84f177987d6 (origin/master, origin/HEAD) Initial commit
```

删除分支

2025年6月9日 16:02

```
liao:niqu_test_66$ git branch -D test
Deleted branch test (was b1cef02).
liao:niqu_test_66$ git log --pretty=oneline --graph --all
* 2fbc2b03c0c602e9ba91a47c48b18c2fa9b54379 (HEAD -> master) 20250609 niqiu modify main.c
* 98ca85631070f3207a992543eea6e5b3f8675c03 20250609 niqiu add main.c
* f9e497e224dbe54ce6ce794d01a6f84f177987d6 (origin/master, origin/HEAD) Initial commit
liao:niqu_test_66$ █
```

工作流程：

写代码之前，先创建一个临时分支，遇事不决，回退到master，再删除临时分支

把test的内容加入master

2025年6月9日 16:07

```
liao:niqui_test_66$ git log --pretty=oneline --graph --all
* f0e66a3fecea3ac872ab0c4510d15d543d079d86 (HEAD -> test) 20250609 test
* 2fbc2b03c0c602e9ba91a47c48b18c2fa9b54379 (master) 20250609 niqiu modify main.c
* 98ca85631070f3207a992543eea6e5b3f8675c03 20250609 niqiu add main.c
* f9e497e224dbe54ce6ce794d01a6f84f177987d6 (origin/master, origin/HEAD) Initial commit
```

合并:

master 主动方

test 被动方

```
liao:niqui_test_66$ git checkout master
Switched to branch 'master'
Your branch is ahead of 'origin/master' by 2 commits.
(use "git push" to publish your local commits)
```

```
liao:niqui_test_66$ git merge test
Updating 2fbc2b0..f0e66a3
```

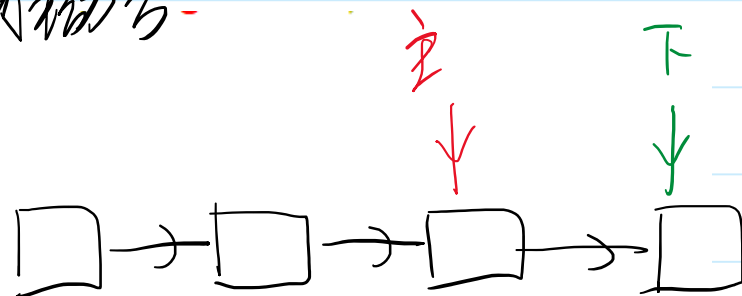
Fast-forward

```
main.c | 1 +
1 file changed, 1 insertion(+)
```

```
liao:niqui_test_66$ git log --pretty=oneline --graph --all
* f0e66a3fecea3ac872ab0c4510d15d543d079d86 (HEAD -> master, test) 20250609 test
```

切换到主动方

合并



当被动方在主动方的正下游时，主动方merge会触发快进

如果不是正下游关系

2025年6月9日 16:11

```
liao:niquiu_test_66$ git log --pretty=oneline --graph --all
* ce0049f6e0b9f22e4c21463e5d04dd76dd2fd06f (HEAD -> test) file2
| * b2a9eee4aa096e936b5e93758cc0b0d4ff7ecb1c (master) file1
|/
* f0e66a3fecea3ac872ab0c4510d15d543d079d86 20250609 test
```

情况1 两个分支改动的文件不一样

```
liao:niquiu_test_66$ git checkout master ← master 是主分支
Switched to branch 'master'
Your branch is ahead of 'origin/master' by 4 commits.
(use "git push" to publish your local commits)
liao:niquiu_test_66$ git merge test ← test 被 merge
Merge made by the 'recursive' strategy.
file2 | 0
1 file changed, 0 insertions(+), 0 deletions(-)
create mode 100644 file2
liao:niquiu_test_66$ git log --pretty=oneline --graph --all
* 5b9befcf86e936f7ada1f2f3217a3be617cde578 (HEAD -> master) Merge branch 'test'
| \
| * ce0049f6e0b9f22e4c21463e5d04dd76dd2fd06f (test) file2
| * | b2a9eee4aa096e936b5e93758cc0b0d4ff7ecb1c file1
|/
* f0e66a3fecea3ac872ab0c4510d15d543d079d86 20250609 test
```

```
graph TD
    Root[ ] --> Test[ ]
    Root --> Master[ ]
    Test --> Merge[ ]
    Master --> Merge
    style Root fill:none,stroke:none
    style Merge fill:none,stroke:none
```

Handwritten notes: "master 是主分支" (master is the main branch) with an arrow pointing to the checkout command. "test 被 merge" (test is merged) with an arrow pointing to the merge command. A red box highlights the merge commit in the graph, with arrows pointing to it from the 'test' and 'master' branches.

merge之后

2025年6月9日 16:17

如何让test也指向新位置

```
liao:niqu_test_66$ git log --pretty=oneline --graph --all
* 5b9befcf86e936f7ada1f2f3217a3be617cde578 (HEAD -> master) Merge branch 'test'
|\
| * ce0049f6e0b9f22e4c21463e5d04dd76dd2fd06f (test) file2
* | b2a9eee4aa096e936b5e93758cc0b0d4ff7ecb1c file1
|/
* f0e66a3fecea3ac872ab0c4510d15d543d079d86 20250609 test
```

```
liao:niqu_test_66$ git checkout test
Switched to branch 'test'
liao:niqu_test_66$ git merge master
Updating ce0049f..5b9befc
Fast-forward
 file1 | 0
 1 file changed, 0 insertions(+), 0 deletions(-)
 create mode 100644 file1
```

两个分支改动了同一个文件

2025年6月9日 16:21

```
liao:niqui_test_66$ git log --pretty=oneline --graph --all
* eb1532444fafe372c74ebc86930f12eb7d6dc495 (HEAD -> master) master test.c
| * a32441b03db34c07d8e95c1b7563feefe2835af1 (test) test main.c
|/
* 5b9befcf86e936f7ada1f2f3217a3be617cde578 Merge branch 'test'
```

```
liao:niqui_test_66$ git merge test
Auto-merging main.c
CONFLICT (content): Merge conflict in main.c
Automatic merge failed; fix conflicts and then commit the result.
liao:niqui_test_66$
```

```
liao:niqui_test_66$ git status
On branch master
Your branch is ahead of 'origin/master' by 7 commits.
(use "git push" to publish your local commits)

You have unmerged paths.
(fix conflicts and run "git commit")
(use "git merge --abort" to abort the merge)

Unmerged paths:
  (use "git add <file>..." to mark resolution)
    both modified:   main.c

no changes added to commit (use "git add" and/or "git commit -a")
```

→ 中断

解决合并冲突的流程

2025年6月9日 16:24

```
main.c
1 #include <stdio.h>
2 int main(){
3     printf("Hello world!\n");
4 <<<<<< HEAD
5     printf("master\n");
6 =====
7     printf("Hello world!\n");
8     printf("test\n");
9 >>>>>> test
10     return 0;
11 }
```

- 1、打开文件 搜索 "<<<<<" 定位到冲突区域
- 2、根据业务需求修改冲突区域
- 3、git add 文件名，用标识某个文件的冲突已经解决
- 4、所有的冲突都被git add了，git commit提交

一个分支修改文件 另一个分支删除文件

2025年6月9日 16:33

```
liao:niqui_test_66$ git log --pretty=oneline --graph --all
* 281758fab0da88c251a1c8a2803d6f56c0baac45 (HEAD -> master) master modify file1
| * d4e18603e1834bc57468612403ce6c5e5cc070e2 (test) test rm file1
|/
* 76e284b3ebf50ddc100f70a2bcfdf4ffc371a3aa fix conflict
..
* f9e497e224dbe54ce6ce794d01a6f84f177987d6 (origin/master, origin/HEAD) Initial commit
liao:niqui_test_66$ git merge test
CONFLICT (modify/delete): file1 deleted in test and modified in HEAD. Version HEAD of file1 left in tree.
Automatic merge failed; fix conflicts and then commit the result.
liao:niqui_test_66$ git status
On branch master
Your branch is ahead of 'origin/master' by 10 commits.
  (use "git push" to publish your local commits)

You have unmerged paths.
  (fix conflicts and run "git commit")
  (use "git merge --abort" to abort the merge)

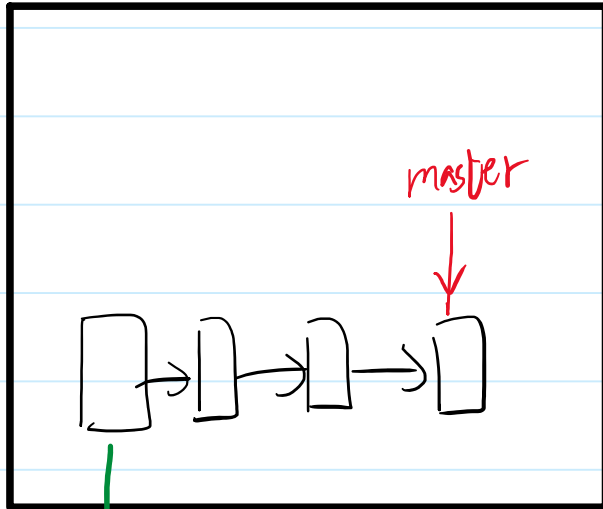
Unmerged paths:
  (use "git add/rm <file>..." as appropriate to mark resolution)
    deleted by them: file1

no changes added to commit (use "git add" and/or "git commit -a")
liao:niqui_test_66$
```

仓库同步

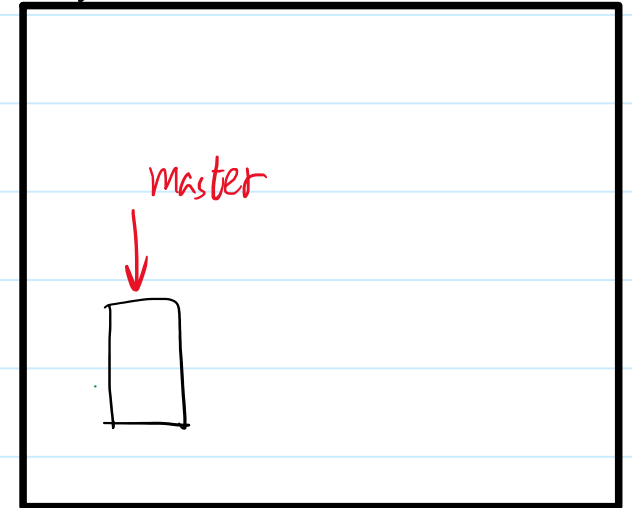
2025年6月9日 16:37

Linux



origin/master

gitee



```
liao:niqu_test_66$ git remote -v
origin  git@gitee.com:liao_zheng_song/niqu_test_66.git (fetch)
origin  git@gitee.com:liao_zheng_song/niqu_test_66.git (push)
```

push可以提交代码

2025年6月9日 16:40

origin 是gitee仓库在Linux仓库视角中的名字

origin/master 是gitee仓库的master分支在Linux仓库当中的引用

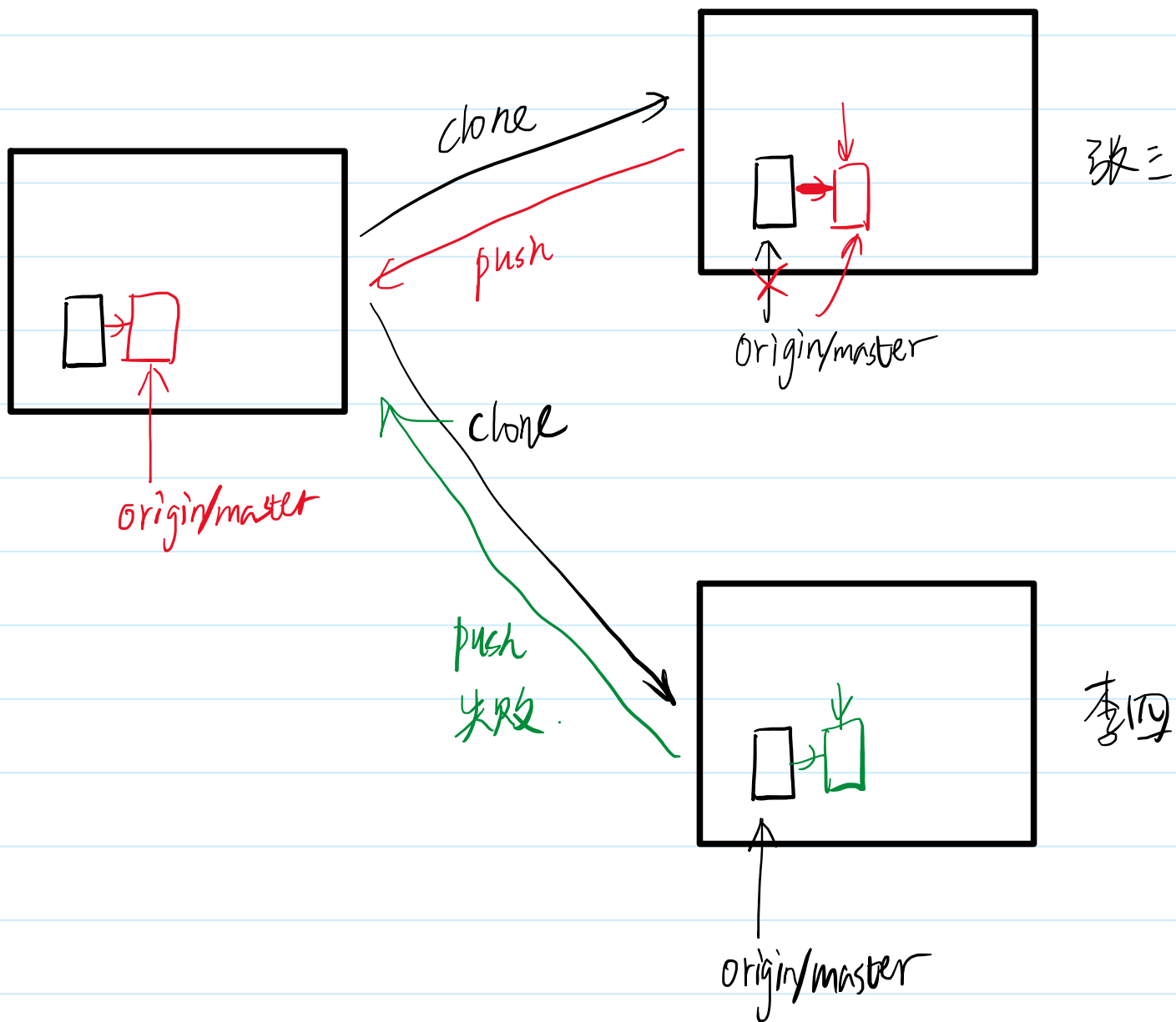
当我们发现自己的master在origin/master的下游，可以执行push操作

```
liao:niqu_test_66$ git push origin master
```

有时候推送会失败

2025年6月9日 16:47

```
liao:niqu_test_66_1$ git log --pretty=oneline --graph --all
* 44adbe1f8a03860fd623ee8cd0025cf0418869a5 (HEAD -> master) 20250609 add test.c
* f9e497e224dbe54ce6ce794d01a6f84f177987d6 (origin/master, origin/HEAD) Initial commit
liao:niqu_test_66_1$ git push origin master
To gitee.com:liao_zheng_song/niqu_test_66.git
 ! [rejected]          master -> master (fetch first)
error: failed to push some refs to 'git@gitee.com:liao_zheng_song/niqu_test_66.git'
hint: Updates were rejected because the remote contains work that you do
hint: not have locally. This is usually caused by another repository pushing
hint: to the same ref. You may want to first integrate the remote changes
hint: (e.g., 'git pull ...') before pushing again.
hint: See the 'Note about fast-forwards' in 'git push --help' for details.
```



git fetch

2025年6月9日 16:50

把origin仓库的最新的master的情况拉取到本地

```
liao:niqui_test_66_1$ git fetch origin master
remote: Enumerating objects: 31, done.
remote: Counting objects: 100% (31/31), done.
remote: Compressing objects: 100% (28/28), done.
remote: Total 30 (delta 18), reused 0 (delta 0), pack-reused 0 (from 0)
Unpacking objects: 100% (30/30), 2.38 KiB | 348.00 KiB/s, done.
From gitee.com:liao_zheng_song/niqui_test_66
 * branch                master      -> FETCH_HEAD
   f9e497e..92fa0e3      master      -> origin/master
liao:niqui_test_66_1$ git log --pretty=oneline --graph --all
* 44adbe1f8a03860fd623ee8cd0025cf0418869a5 (HEAD -> master) 20250609 add test.c
| * 92fa0e326603c17062abfef850450c393ea01a3f (origin/master, origin/HEAD) fix conflict
| | \
| | * d4e18603e1834bc57468612403ce6c5e5cc070e2 test rm file1
| * | 281758fab0da88c251a1c8a2803d6f56c0baac45 master modify file1
| | /
| * 76e284b3ebf50ddc100f70a2bcfd4fffc371a3aa fix conflict
| | \
```

origin/master 不是真正的分支

2025年6月9日 17:13

origin/master是origin仓库在本地的一个引用，它不是真正的分支

不要checkout 到origin/master里面

origin/master可以作为merge的被动方

```
liao:niqu_test_66_1$ git merge origin/master
Merge made by the 'recursive' strategy.
 file1 | 1 +
 file2 | 0
main.c | 7 +++++++
3 files changed, 8 insertions(+)
create mode 100644 file1
create mode 100644 file2
create mode 100644 main.c
liao:niqu_test_66_1$ git log --pretty=oneline --graph --all
*    a882de7db842d2dcb6aa30620156ff259d3d64ee (HEAD -> master) Merge re
||\
```

merge之后可以再push

push失败的流程

2025年6月9日 17:16

1、git fetch origin master

2、git merge origin/master

git merge origin master

3、git push origin master

小组的流程

2025年6月9日 17:18

0、使用自己的仓库练习git指令

1、组长建一个仓库

2、邀请组员作为开发者or管理员

 廖正松/niqui_test_66 

Watching 1

Star 0

Fork 0

 代码

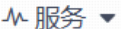
 Issues 0

 Pull Requests 0

 Wiki

 统计

 流水线

 服务

 管理

仓库设置

仓库镜像管理

开源评估指数

仓库成员管理

所有1

管理员1

开发者0

开发者管理

邀请用户

「当前仓库暂无开发者权限成员」

?





流程

2025年6月9日 17:20

- 3、新建临时分支，checkout到临时分支，写代码，add再commit
 - 4、测试，测试通过之后，用master去merge临时分支，删除临时分支
 - 5、git push origin master, 其他同学可以git fetch origin master
- (不用随便使用git merge/git pull, 使用这些指令必须确保工作树是干净的)

尽量不要用没教过的指令

2025年6月9日

17:25

~~git push --force~~

~~git reset~~

如果要取消一个版本做的事情, git revert

尽量不要回溯

2025年6月9日 17:30

```
liao:niqu_test_66$ git checkout 76e28  
Note: switching to '76e28'.
```

← 危险状态

```
liao:niqu_test_66$ git branch temp
```

```
liao:niqu_test_66$ git checkout temp  
Switched to branch 'temp'
```

总结一下

2025年6月9日 17:32

1、单机

`git add --> git commit`

2、分支

`git branch`

`git checkout`

`git merge --> 尽量快进`

3、多人合作

`git push origin master`

- 先fetch 再merge

- `git fetch origin master`

- `git merge origin/master`

- pull

- `git pull origin master`